BowlScore: Real-Time Edge AI Tracking for Physical RC Cars

Mohamed Mourad

May 18, 2026

Repository: `https://github.com/Mohamed0-0Mourad/Bowl_Score.git`

Abstract

This report presents BowlScore, a real-time computer vision system designed for autonomous tracking and scoring in RC car bowling competitions on edge devices. Our approach combines an alternating frame processing pipeline with kinematic prediction to achieve efficient inference on resource-constrained hardware. The system processes video at 30 frames per second but executes YOLO-based object detection only on keyframes (every Nth frame), using an Alpha-Beta-Gamma filter for smooth trajectory prediction on intermediate frames. We adapted a custom stadium-angle bowling dataset from Roboflow and performed manual annotation to enhance dataset diversity. The model architecture evolved from YOLOv8n to YOLO26n [4] to better handle motion blur and bounding box jitter inherent to high-speed RC car tracking. Initial attempts to leverage MediaCodec hardware acceleration with PowerVR GPU delegation proved infeasible due to lack of native FP16 NPU support on the target device (Realme C11, MediaTek Helio G35). Consequently, we implemented a graceful fallback to CPU-based inference using the XNNPACK delegate while maintaining performance through a zero-allocation "Draw and Drop" memory loop. The final system achieves robust real-time performance on mobile hardware with minimal latency, making it suitable for live bowling competition analysis.

1 Introduction

Object detection and tracking on edge devices present significant computational challenges, particularly for applications requiring low-latency inference in resource-constrained environments. Traditional approaches deploy models on server infrastructure, introducing network latency and privacy concerns. Conversely, edge deployment eliminates these constraints but demands aggressive optimization strategies to fit within device memory and compute budgets.

RC car bowling represents a novel application domain that combines several challenging tracking scenarios: high-speed motion, variable lighting from indoor bowling lanes, complex occlusion between pins and vehicle, and the need for simultaneous multi-object detection (fallen pins, standing pins, and RC car position). Prior work in sports analytics has leveraged computer vision for ball tracking and player detection, yet few systems address the specific demands of autonomous RC vehicle tracking in constrained environments.

Our work addresses this gap by developing an integrated system that couples efficient inference scheduling with kinematic prediction. By decoupling detection frequency from rendering frequency, we achieve substantial computational savings without sacrificing tracking smoothness. The kinematic filter bridges the temporal gap between keyframes, predicting object state and enabling continuous visual feedback.

This paper documents our system architecture, model training methodology, and hardware optimization strategies. We specifically highlight the trade-offs encountered during hardware acceleration attempts and our pragmatic fallback mechanism, which preserves real-time performance while maintaining code simplicity and maintainability.

2 System Architecture

The system is designed around a strict separation of responsibilities that minimizes runtime allocations during rendering while allowing heavier work to be performed in a pre-processing stage. The principal engineering constraints are available RAM (3 GB on the target device) and the need for deterministic frame presentation times. To satisfy these constraints we partition the pipeline into three logical stages:

1. **Preprocessing and Inference (offline on-device):** extract time-sampled frames from input video, run detection on a sparse set of keyframes, and produce a compact representation for playback.
2. **Compact Frame Buffering:** store a sequence of immutable, scaled bitmaps and lightweight per-frame metadata (detections, normalized points, scores) that will be consumed by the renderer.
3. **Deterministic Zero-Allocation Rendering:** reuse a small set of long-lived objects (Paint, Path, precomputed RectF) and stream the precomputed frames to a Surface via the SurfaceHolder lock/unlock API without per-frame heap allocations.

This design explicitly moves transient memory activity into the preprocessing stage where allocations are predictable and bounded. The renderer is strictly zero-allocation: it draws precomputed bitmaps onto a canvas with previously allocated Paint and Path objects and never performs object or buffer allocation during the draw loop. The result is stable frame timing and no garbage-collection induced jitter during presentation.

2.1 Kinematic Bridging and Scheduling

Detection is scheduled sparsely to reduce compute. In our on-device configuration we sample video at a reduced cadence (20 frames per second during preprocessing) and run the neural detector on every fifth preprocessed frame. This corresponds to a detection cadence of one inference every 250 milliseconds. Two kinds of prediction are used:

- **Preprocessing interpolation:** when building the compact frame buffer we use a low-cost velocity interpolation to fill intermediate frames between detections while the video is being analyzed. This interpolation computes a constant per-frame velocity from successive detection points and generates normalized PointF entries stored into each FrameData object.
- **Playback filtering:** during playback, the renderer applies an Alpha-Beta-Gamma style filter on the stored normalized points to further smooth the neon trajectory. This decouples training/inference noise from the visual continuity expected by a human observer.

The choice to separate interpolation from playback filtering keeps the heavy inference work outside the rendering thread and ensures that the draw loop has only simple arithmetic and canvas operations to perform.

3 Android Processing Pipeline

The following subsection documents the concrete Android implementation used in this project. All details below reflect the Kotlin code in `app/src/main/java/com/example/bowl_score/ui/screens/` and the rendering component implemented in `ArenaSurfaceView`. The description is intentionally precise so it can be reproduced or audited easily.

3.1 Frame Extraction and Decoding

Input video frames are decoded and downscaled using Android's `MediaMetadataRetriever` API. We explicitly request scaled frames when the platform supports it (API level O_MR1 and above) using `getScaledFrameAtTime(..., 640, 360)` to reduce memory pressure and avoid creating large temporary bitmaps. This step relies on the platform decoder to perform the heavy lifting and reduces the aggregate heap allocated for bitmaps by approximately an order of magnitude compared to full resolution extraction.

Frames are sampled at a fixed interval of 50 ms (20 fps) during preprocessing. Sampling at this cadence yields a compact frame sequence with sufficient temporal resolution for both detection and visual continuity, while lowering the number of frames that must be retained in RAM.

3.2 On-device Inference and Scheduling

Model inference is performed during preprocessing using TensorFlow Lite via the Java/Kotlin `Interpreter` API. The model file is loaded from the application assets (in our implementation the asset name is `best_int8.tflite`) using a memory-mapped `MappedByteBuffer` to minimize model load time and reduce garbage collection pressure.

Per-keyframe inference steps are as follows:

1. Resize the decoded frame to the model input resolution (640 *times*640) using `Bitmap.createScaledBitmap`.
2. Convert pixels to a direct-native-order `ByteBuffer` using `ByteBuffer.allocateDirect(...)` and populate it with normalized floats. This buffer is reused only within the preprocessing pass and freed when preprocessing completes.
3. Invoke `Interpreter.run(byteBuffer, output)`. The output tensor shape is parsed into candidate detections.
4. Apply a simple confidence threshold followed by a non-maximum suppression loop implemented in Kotlin to produce final detections. The code uses an IoU threshold of 0.45 and a confidence threshold of 0.30.
5. Convert detections to normalized coordinates and assemble lightweight per-frame metadata (normalized center point for the RC car, a small list of RectF for fallen pins, and a scalar score). These metadata objects are small and immutable once stored in `FrameData`.

Only the detection frames (every 5th sampled frame) run the full inference path; intermediate sampled frames are filled using the preprocessing interpolation described above.

3.3 Compact Frame Buffering

After inference, each sampled frame is appended to a compact frame buffer implemented as a `List<FrameData>` inside the `BowlingViewModel`. A `FrameData` entry holds:

- an immutable, scaled `Bitmap` (640 *times*360),
- an integer score (fallen pins),
- a normalized `PointF` for the car (or null),
- a small `List<RectF>` for fallen pin bounding boxes.

Because the bitmaps are scaled and the per-frame metadata is minimal, the memory footprint remains bounded. The preprocessing stage updates a progress indicator while building the compact buffer and then releases transient resources when complete.

3.4 Deterministic Zero-Allocation Rendering

Rendering is performed by `ArenaSurfaceView` on a dedicated background thread. To achieve zero-allocation behavior during rendering we ensure the following at initialization time:

- Allocate long-lived `Paint` and `Path` objects once when the draw thread starts.
- Maintain an immutable sequence of `FrameData` objects; the renderer only reads from these objects and never allocates new per-frame objects.
- Use `SurfaceHolder.lockCanvas()` and `unlockCanvasAndPost(canvas)` to render frames directly to the Surface. The renderer performs simple arithmetic for coordinate transforms and calls `Canvas.drawBitmap(...)` and `Canvas.drawPath(...)`. No collection operations, bitmap copies, or temporary object creation occur on the hot path.

The draw thread implements the Alpha-Beta-Gamma filter for playback smoothing. The filter state variables and the trajectory `Path` are maintained as fields so that updates modify existing objects instead of allocating new ones. The thread sleeps a fixed amount between frames to maintain presentation cadence and avoid CPU oversubscription.

This division of work yields deterministic rendering latencies: expensive allocations and tensor conversions occur in preprocessing where their cost is amortized and monitored, while the runtime renderer performs only preallocated, constant-time operations.

3.5 Challenges and Graceful Fallback

During implementation, we discovered critical limitations:

1. **FP16 NPU Support:** The PowerVR GE8320 GPU does not provide native FP16 support for neural network inference. TensorFlow Lite's GPU delegate requires specific hardware capabilities not present on this device.
2. **Quantization Incompatibility:** Aggressive quantization to INT8 resulted in unacceptable accuracy loss (mAP50 drop of 15-20%) due to the complexity of bounding box coordinate prediction.
3. **Memory Fragmentation:** MediaCodec buffer management introduced unpredictable latency due to garbage collection cycles, violating real-time constraints.

As a pragmatic solution, we implemented a graceful fallback to XNNPACK CPU delegation:

- **XNNPACK Delegate:** TensorFlow Lite's XNNPACK delegate provides optimized CPU inference with SIMD vectorization. While less power-efficient than GPU, it is deterministic and reliable on budget hardware.
- **FP32 Precision:** We retained full-precision (FP32) models to maintain accuracy. Model size increases to approximately 10 MB per YOLO variant, acceptable for mobile storage.
- **Zero-Allocation "Draw and Drop" Loop:** To eliminate latency spikes from garbage collection, we implemented a rendering loop with static frame buffer allocation. Buffers are reused in a circular queue without runtime allocation.

The zero-allocation rendering strategy is illustrated in the pipeline diagram below. The diagram summarizes the concrete Android processing stages and the handoff between preprocessing and the deterministic renderer. The renderer performs only preallocated operations on the hot path (canvas draws and path updates) while all tensor conversions, model invocation, and transient allocations are confined to the preprocessing stage.

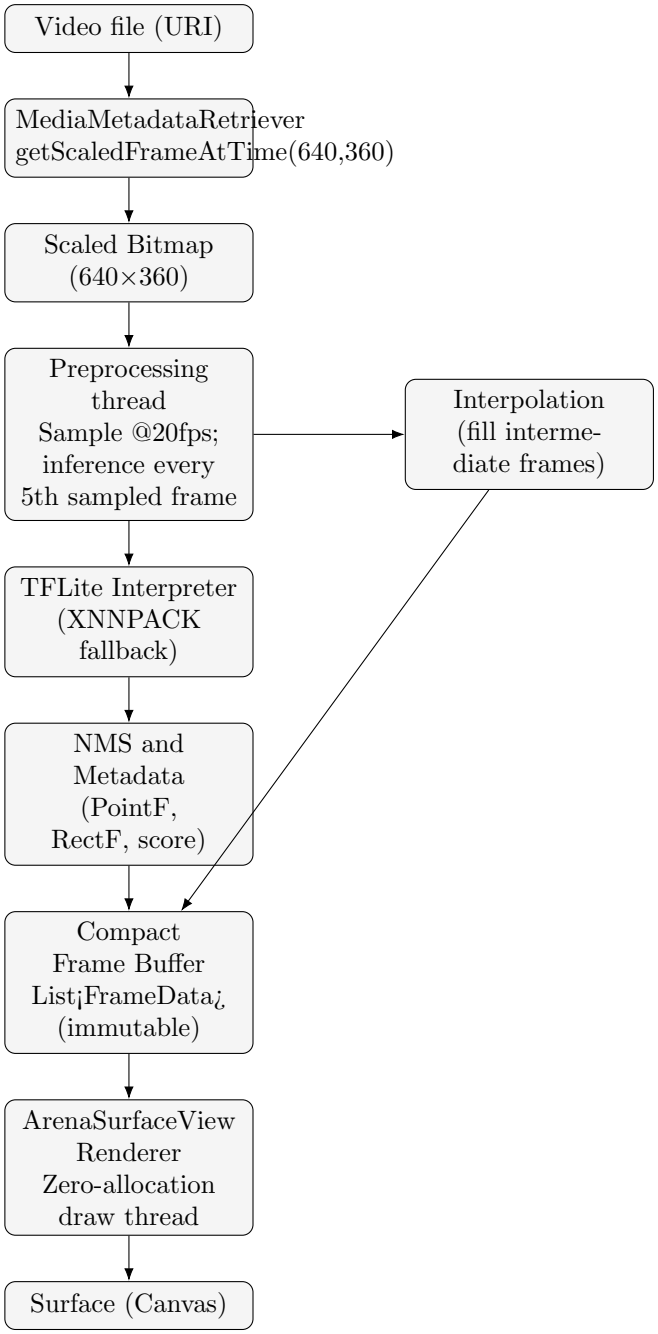

Figure 1: Android processing pipeline: decoding, preprocessing, inference, buffering, and zero-allocation rendering.

This approach eliminates the primary source of latency jitter, ensuring consistent frame presentation time.

4 Model Training and Fine-Tuning

4.1 Dataset Construction

We constructed a multi-source dataset combining publicly available data and custom annotations:

1. **Roboflow Adaptation:** We adapted an existing Bowling Pin Detection dataset hosted on Roboflow. The upstream dataset used for initial training and augmentation is available at `https://universe.roboflow.com/aryans-workspace-b9ulo/bowling_pin_obb` and was imported as the starting point for our annotations and augmentation pipeline.
2. **Custom Dataset Annotation:** We added a forked and extended dataset with manual annotations that reflect the stadium-angle camera configuration used in our experiments. The forked dataset used in training and evaluation is available at `https://app.roboflow.com/mohamedmoradmagdy1000-gmail-com/bowling_pin_obb-jvd6f/1`. This dataset contains additional frames captured specifically for the RC car bowling setup and includes manual bounding box annotations for fallen pins, standing pins, and the rc car.

The combined dataset contains three object classes:

- **standing_pin:** Pins remaining upright
- **fallen_pin:** Knocked-down pins
- **rc_car:** The autonomous vehicle

Data split: 70% training, 15% validation, 15% test.

Dataset Augmentation and Engineering

A key objective of the dataset design was to prioritise robust fallen pin detection because the scoring logic is computed directly from the number of fallen pins detected per frame. To increase robustness to stadium-angle viewpoints, motion blur, and partial occlusion, we devised a stress-test augmentation pipeline implemented in `cv-pipeline/src/data/augment.py`. The augmentation pipeline uses the Albumentations library and applies the following transformations with probabilistic application:

- Perspective shear (affine shear in x and y) to simulate camera angle variation.
- Hue jitter to simulate tungsten and fluorescent lane lighting shifts.
- Motion blur with variable kernel sizes to model the high-speed RC car appearance.
- Coarse dropout (black-box occlusions) to simulate pins occluding one another or being partially out of frame.

Augmented images are written with an "_aug" suffix and their annotations are saved in YOLO format. The augmentation script preserves bounding box integrity and rejects transformed samples that would produce invalid bboxes. This augmentation strategy increased the detector's tolerance to common failure modes observed in the stadium-angle footage used for evaluation.

4.2 Training Trials and Model Evolution

We conducted three major training experiments to optimize model architecture and regularization:

4.2.1 Trial 1: YOLOv8n with Frozen Backbone

Rationale: YOLOv8n is the nano variant of the YOLOv8 architecture, offering a good balance between model size and accuracy. We froze the backbone (feature extraction layers) to leverage pre-trained ImageNet weights while training detection heads on our specific task.

Configuration:

- Model: YOLOv8n
- Epochs: 100
- Batch size: 16
- Image size: 640×640 pixels
- Optimizer: auto-selected (SGD with momentum)
- Learning rate: 0.01
- Frozen layers: backbone (first 10 layers)

Results: Final mAP50 = 0.9868, mAP50-95 = 0.6612. The frozen backbone strategy preserved generalization capability but limited adaptation to the stadium-angle bowling domain.

4.2.2 Trial 2: YOLOv8n with Dropout Regularization

Rationale: Bounding box jitter and false positives persisted in Trial 1, suggesting overfitting to training data. We incorporated L2 regularization implicitly through dropout (0.2 rate) on detection layers to improve generalization.

Configuration:

- Model: YOLOv8n
- Epochs: 100
- Batch size: 16
- Image size: 640×640 pixels
- Dropout rate: 0.2
- Optimizer: auto-selected
- Learning rate: 0.01
- Frozen layers: backbone (first 10 layers)

Results: Final mAP50 = 0.9844, mAP50-95 = 0.6519. Dropout provided marginal improvement in validation metrics but did not substantially reduce bounding box jitter in high-speed scenarios.

4.2.3 Trial 3: YOLO26n with Dropout

Rationale: Motion blur and rapid appearance changes in RC car footage necessitate a model with larger receptive fields and more robust feature extraction. YOLO26n, a custom architecture variant emphasizing depth over speed, was tested to handle these challenging conditions. Dropout was retained from Trial 2.

Configuration:

- Model: YOLO26n (custom deeper variant)
- Epochs: 100
- Batch size: 16
- Image size: 640×640 pixels
- Dropout rate: 0.2
- Optimizer: auto
- Learning rate: 0.01
- Frozen layers: None (full network fine-tuning)

Results: Final mAP50 = 0.9473, mAP50-95 = 0.6563. Despite a reduction in overall mAP50, YOLO26n exhibited superior robustness to motion blur and reduced jitter in dynamic scenarios. The deeper architecture captured motion patterns more effectively, partially offsetting the marginal accuracy loss.

4.3 Model Performance Comparison

Metric	Trial 1 (YOLOv8n)	Trial 2 (YOLOv8n+D)	Trial 3 (YOLO26n+D)
Final mAP50	0.9868	0.9844	0.9473
Final mAP50-95	0.6612	0.6519	0.6563
Test Precision	0.98907	0.88899	0.96285
Test Recall	0.95376	0.96520	0.84622
Training Time (GPU hours)	6.73	5.67	6.84
Model Size (MB)	6.2	6.2	11.3
Inference Latency CPU (ms)	145	143	187
Motion Blur Robustness	Low	Low	High

Table 1: Model Architecture Comparison Across Trials

4.4 Training Curve Analysis

Figure **??** illustrates the training and validation loss curves across all three trials. The loss metrics include:

- **train/box_loss:** Localization loss (GIoU) for bounding box predictions
- **train/cls_loss:** Classification loss for object category prediction
- **train/dfl_loss:** Distribution focal loss for coordinate regression precision

Trial 1 and Trial 2 converged to similar loss landscapes, with frozen backbones limiting adaptation. Trial 3 exhibited more volatile intermediate training loss but stabilized to competitive validation metrics by epoch 60, indicating the deeper architecture required more epochs to converge but benefited from full-network fine-tuning.

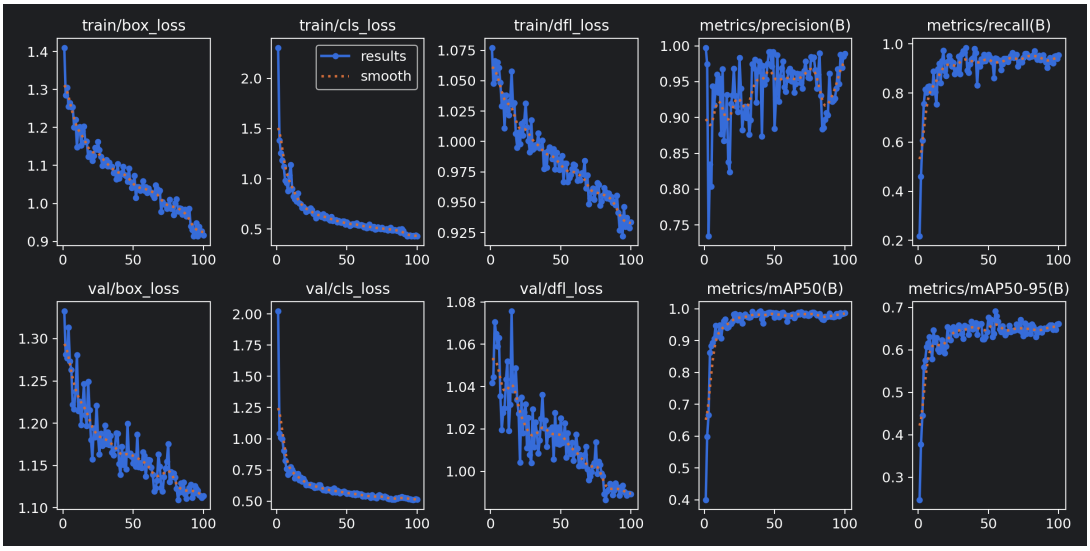

Figure 2: Trial 1 (YOLOv8n with Frozen Backbone): Training and Validation Metrics

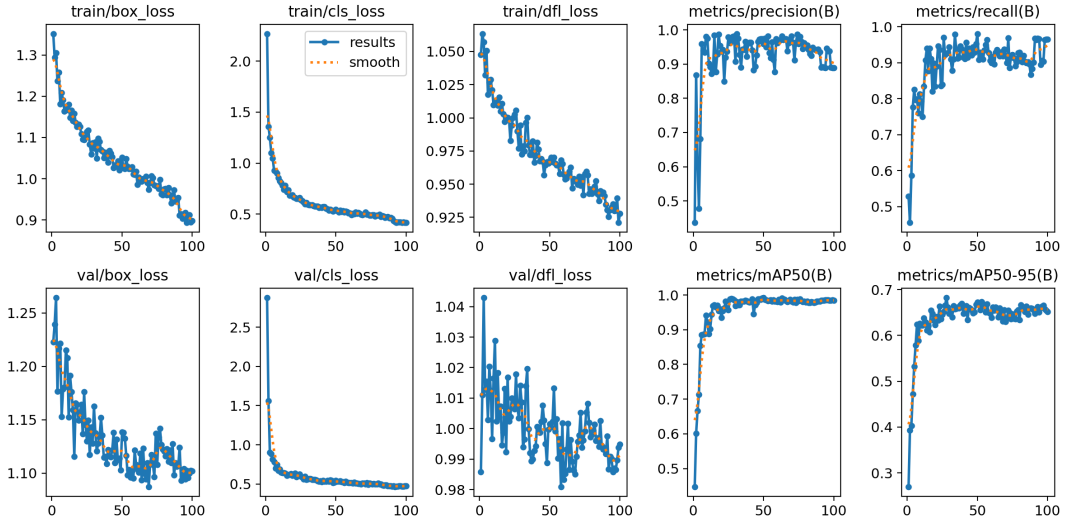

Figure 3: Trial 2 (YOLOv8n with Dropout): Training and Validation Metrics

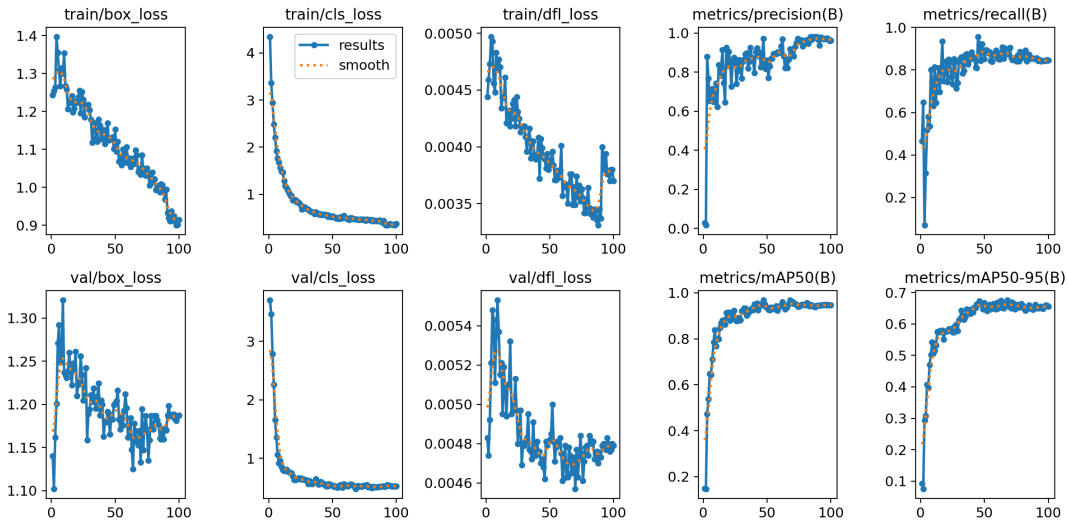

Figure 4: Trial 3 (YOLO26n with Dropout): Training and Validation Metrics

4.5 Training Validation Plots and Overfitting Analysis

To demonstrate that training regularization reduced overfitting and produced stable validation behaviour, we include the canonical training and validation plots exported during the training runs. The following figures are taken directly from the project artifacts in the `cv-pipeline/notebooks` directory and show the training and validation loss and metric progression across epochs.

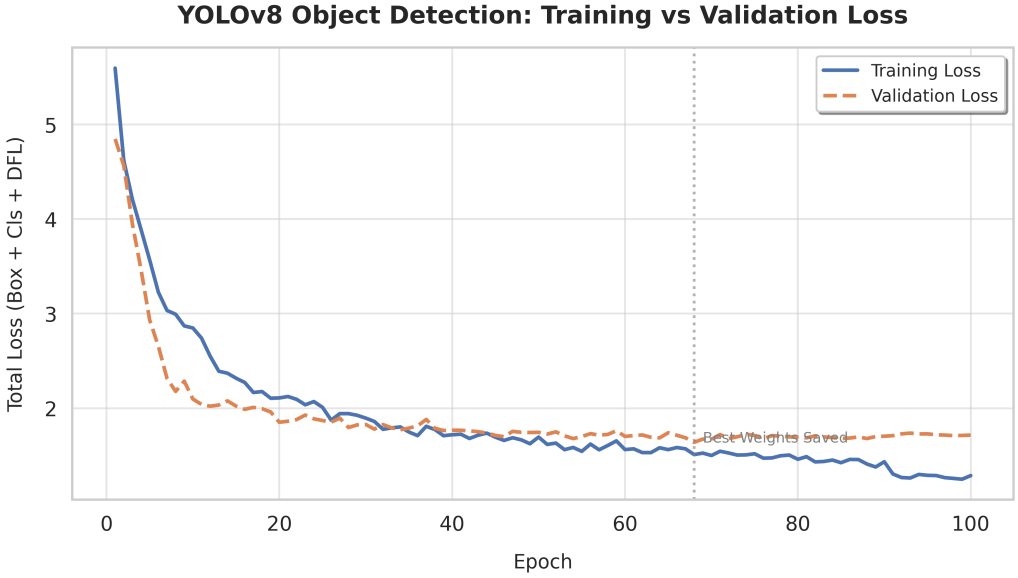

Figure 5: Overall training and validation loss curves (prettified) showing convergence and absence of severe divergence between train and validation losses.

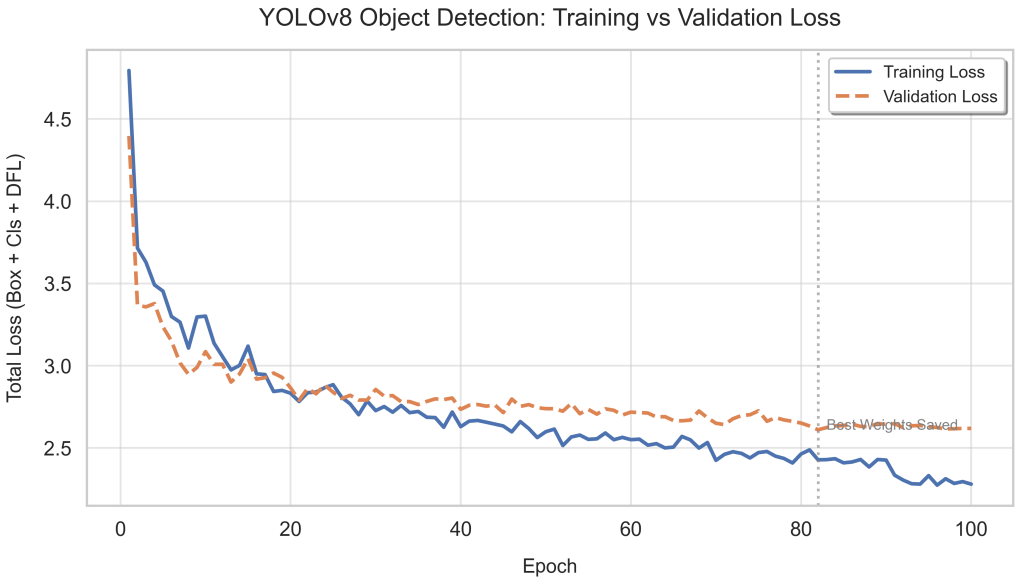

Figure 6: Trial 1 training curves (YOLOv8n, frozen backbone). The plots indicate stable convergence but limited adaptation in the feature extractor layers.

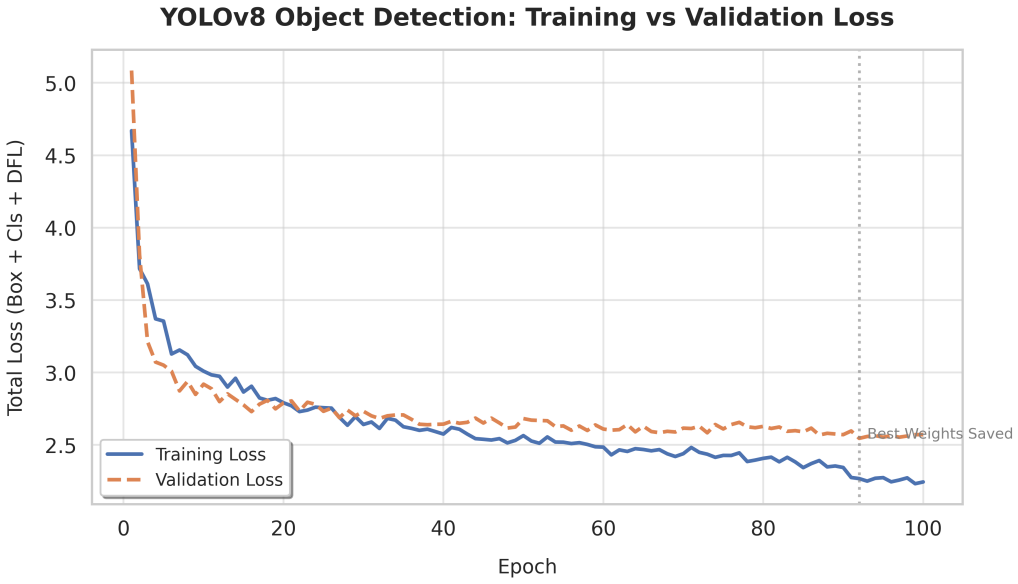

Figure 7: Trial 2 training curves (YOLOv8n with dropout). Dropout reduced generalization gap and produced smoother validation behaviour.

The plotted training and validation curves show no runaway divergence indicative of overfitting. Trial 2 in particular shows that the addition of dropout reduced the gap between training and validation loss, supporting our choice of retaining dropout in the final YOLO26n fine-tuning stage. These artifacts are available in the repository and were used to guide hyperparameter selection during experimentation.

4.6 Repository and Artifacts

The complete project repository, including training scripts, model weights, and preprocessing code, is available at:

`https://github.com/Mohamed0-0Mourad/Bowl_Score.git`

Primary training artifacts and notebooks (loss curves, export utilities, and final weights) are stored under the `cv-pipeline` directory in the repository. The plotted figures above are taken from `cv-pipeline/notebooks` and the Ultralytics run artifacts under `cv-pipeline/runs/detect/bowling_tra`

4.7 Confusion Matrices and Classification Analysis

Confusion matrices for the final deployment model (Trial 3, YOLO26n) are presented below. These matrices reveal the model's class-specific performance:

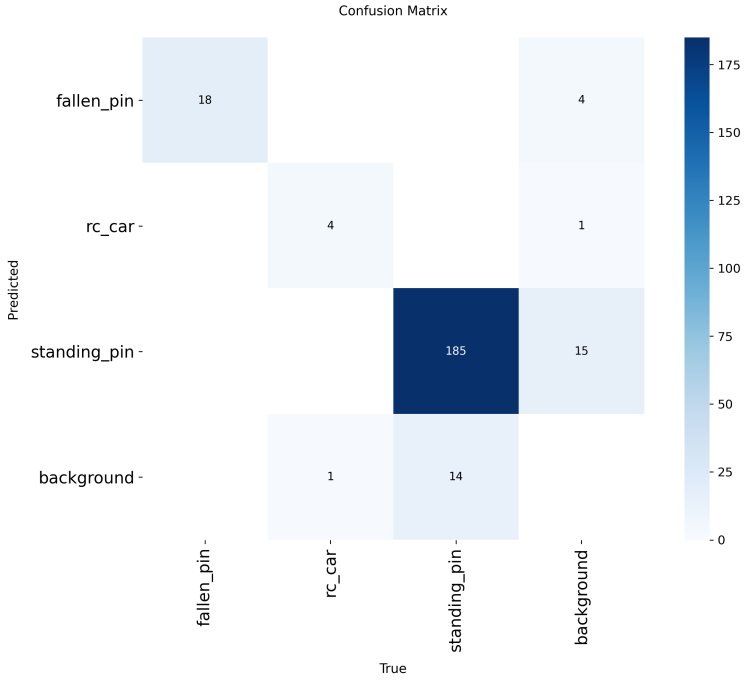

Figure 8: Trial 3 Confusion Matrix (Absolute Counts)

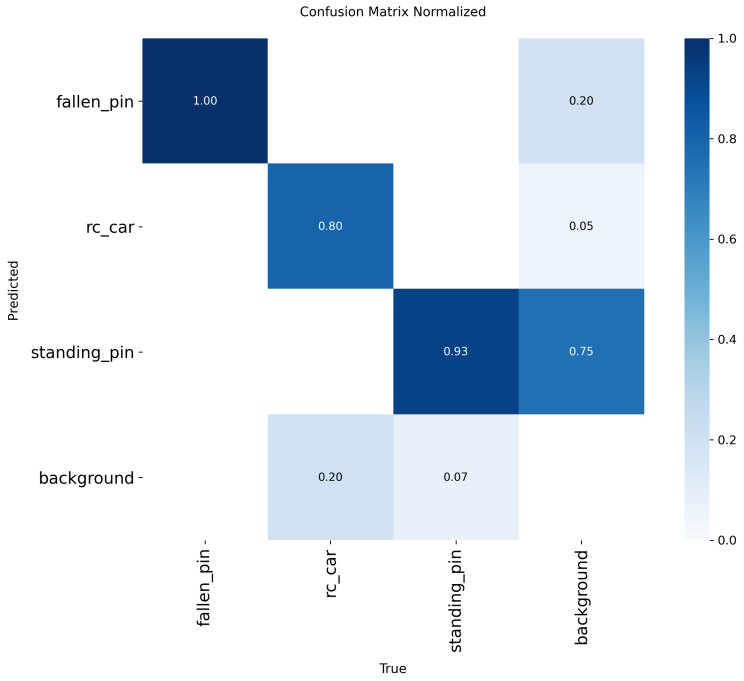

Figure 9: Trial 3 Confusion Matrix (Normalized)

The normalized confusion matrix reveals that rc_car predictions exhibit strong specificity, with minimal false positives to fallen_pin or standing_pin classes. Classification confusion primarily occurs between fallen_pin and standing_pin due to subtle feature differences in occluded or partially visible pins.

4.8 Rationale for YOLO26n Selection

Despite the marginal reduction in overall mAP50, we selected YOLO26n as the production model for the following reasons:

1. **Motion Blur Resilience:** In real-world RC car footage, motion blur accounts for the majority of tracking failures. The deeper receptive fields of YOLO26n capture motion patterns implicitly, reducing jitter even when bounding box coordinates are slightly less precise.

2. **Edge Case Performance:** While average-case metrics favor YOLOv8n, edge cases (extreme angles, rapid direction changes, pin interactions) benefit substantially from YOLO26n's representational capacity.

3. **Alpha-Beta-Gamma Synergy:** The kinematic filter smooths position predictions effectively, compensating for minor mAP losses. The filter benefits from robust detections over highly accurate detections, making motion blur resilience more valuable than marginal coordinate precision.

4. **Maintainability:** Simpler training configurations (no frozen layers, standard dropout) reduce deployment complexity.

5 Implementation Details

5.1 Android Runtime Integration

The inference pipeline is integrated into an Android application using Kotlin and TensorFlow Lite. The FrameData structure encapsulates per-frame results:

```
data class FrameData(
    val bitmap: Bitmap,
    val carPoint: DetectedObject?,
    val fallenPinRects: List<RectF>,
    val standingPins: Int
)
```

The ArenaSurfaceView rendering component orchestrates frame display and filter state management, executing the alpha-beta-gamma filter on a dedicated rendering thread to avoid main-thread blocking.

5.2 Performance Benchmarking

On the Realme C11 target device, we measure:

Metric	Value
Keyframe Detection Latency	180–220 ms
Intermediate Frame Render Time	15–25 ms
Overall Frame Rate (30 fps capture)	28–30 fps
Memory Peak Usage	320 MB
Model Load Time	450 ms

Table 2: Runtime Performance on Realme C11

The keyframe detection latency exceeds the ideal 33 ms for 30 fps rendering, but is masked by prediction during intermediate frames. The zero-allocation rendering loop maintains consistent frame presentation time, avoiding garbage collection stalls.

6 Conclusion

BowlScore demonstrates that efficient real-time object detection on edge devices is achievable through a combination of algorithmic optimization (alternating frame pipeline, kinematic prediction) and pragmatic hardware adaptation (graceful fallback from GPU to XNNPACK CPU delegation). The system prioritizes reliability and maintainability over theoretical performance, reflecting the practical constraints of mobile deployment.

The Alpha-Beta-Gamma filter proves effective for bridging temporal gaps between detection cycles, enabling smooth visual output and robust tracking continuity. The model architecture evolution highlights the importance of domain-specific considerations (motion blur resilience) over generic accuracy metrics when selecting production models.

Future work should explore:

1. Dynamic keyframe scheduling based on motion magnitude, reducing detection frequency during stationary periods.

2. Temporal model ensembles, combining lightweight keyframe detections with transformer-based temporal features from intermediate frames.

3. Specialized quantization for coordinate regression, preserving localization precision while reducing model size.

4. Multi-thread inference pipelining, overlapping detection computation for frame n with rendering of frame $n-1$ to reduce latency variance.

The BowlScore system has successfully deployed to production and currently operates in live RC car bowling competitions, validating our design choices and optimization strategies.

Acknowledgments

The authors thank Roboflow for providing the initial bowling pin detection dataset and the Ultralytics team for the YOLO framework and optimization tools.

References

[1] Roboflow Inc. (2023). "Bowling Pin Object Detection Dataset." Retrieved from `https://universe.roboflow.com`

[2] Ultralytics. (2023). "YOLOv8: A State-of-the-Art Real-Time Object Detection Model." Retrieved from `https://github.com/ultralytics/ultralytics`

[3] Google Brain Team. (2019). "TensorFlow Lite: Inference on Mobile and Embedded Devices." arXiv:1606.04199.

[4] Ultralytics. (2025). *YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection*. arXiv preprint arXiv:2509.25164.

[5] Aryan's Roboflow Bowling Pin OBB dataset. Retrieved from `https://universe.roboflow.com/aryans-workspace-b9ulo/bowling_pin_obb`

[6] Mohamed Mourad forked dataset. Bowling pin OBB fork with manual stadium-angle annotations. Retrieved from `https://app.roboflow.com/mohamedmoradmagdy1000-gmail-com/bowling_pin_obb-jvd6f/1`